

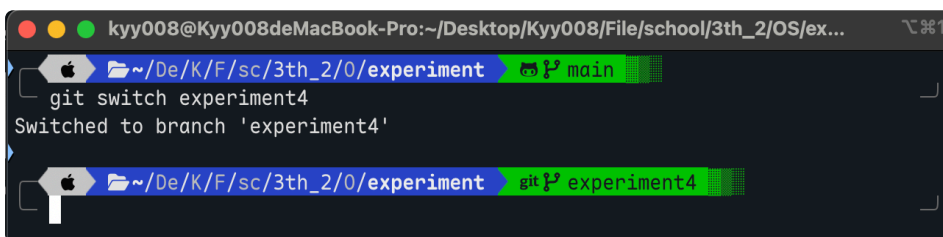
2026 春《操作系统》课程实验报告

实验 4

23301037 客昱阳

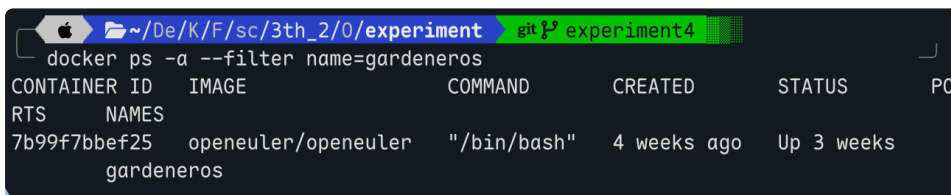
一.准备工作

- 将分支切换到实验四所需的 `experiment4` 分支



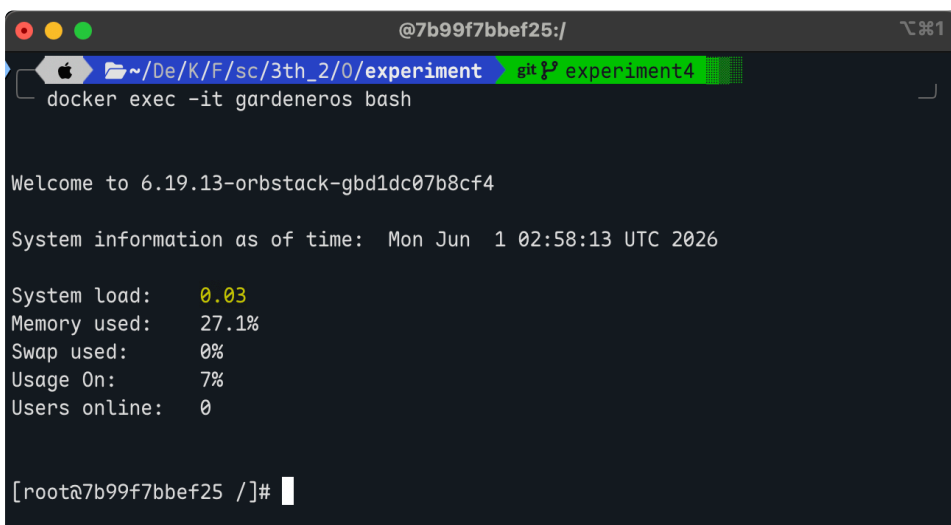
```
kyy008@Kyy008deMacBook-Pro:~/Desktop/Kyy008/File/school/3th_2/OS/ex...  
git switch experiment4  
Switched to branch 'experiment4'
```

- 检查一下上次环境配置的 docker 容器已经启动



```
docker ps -a --filter name=gardeneros  
CONTAINER ID   IMAGE             COMMAND                  CREATED        STATUS        PORTS  
RTS           NAMES  
7b99f7bbef25   openeuler/openeuler  "/bin/bash"            4 weeks ago   Up 3 weeks  
gardeneros
```

- 进入容器



```
@7b99f7bbef25:/  
docker exec -it gardeneros bash  
  
Welcome to 6.19.13-orbstack-gbd1dc07b8cf4  
  
System information as of time: Mon Jun 1 02:58:13 UTC 2026  
  
System load:    0.03  
Memory used:    27.1%  
Swap used:      0%  
Usage On:       7%  
Users online:   0  
  
[root@7b99f7bbef25 /]#
```

二.实验步骤

1.实现应用程序

(1)应用程序的放置

- 多道程序操作系统中，每个应用程序需要加载到不同的内存位置。因此先在 `user` 目录下创建 `build.py`，用于在编译每个应用程序前临时修改 `linker.ld` 中的 `BASE_ADDRESS`，让不同应用程序拥有不同的起始地址。

Bash

```
1 cd /mnt
2 vim user/build.py
```

写入以下内容



```
vim user/build.py
[root@7b99f7bbef25 mnt]# cat user/build.py
import os

base_address = 0x8040000
step = 0x20000
linker = 'src/linker.ld'

app_id = 0
apps = os.listdir('src/bin')
apps.sort()
for app in apps:
    app = app[:app.find('.')]
    lines = []
    lines_before = []
    with open(linker, 'r') as f:
        for line in f.readlines():
            lines_before.append(line)
            line = line.replace(hex(base_address), hex(base_address + step * app_id))
            lines.append(line)
    with open(linker, 'w+') as f:
        f.writelines(lines)
    os.system('cargo build --bin %s --release' % app)
    print('[build.py] application %s start with address %s' % (app, hex(base_address + step * app_id)))
    with open(linker, 'w+') as f:
        f.writelines(lines_before)
    app_id = app_id + 1
[root@7b99f7bbef25 mnt]#
```

- 修改 `user/Makefile`，让用户程序编译时调用刚才创建的 `build.py`。这样每个应用程序都会在不同的地址下重新链接，适应多道程序操作系统的加载方式。

Python

```
1 vim user/Makefile
```

将文件修改为以下内容：

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim user/Makefile
[root@7b99f7bbef25 mnt]# cat user/Makefile
TARGET := riscv64gc-unknown-none-elf
MODE := release
APP_DIR := src/bin
TARGET_DIR := target/${TARGET}/${MODE}
APPS := $(wildcard ${APP_DIR}/*.rs)
ELFS := $(patsubst ${APP_DIR}/%.rs, ${TARGET_DIR}/%, ${APPS})
BINS := $(patsubst ${APP_DIR}/%.rs, ${TARGET_DIR}/%.bin, ${APPS})

OBJDUMP := rust-objdump --arch-name=riscv64
OBJCOPY := rust-objcopy --binary-architecture=riscv64

elf: ${APPS}
    @python3 build.py

binary: elf
    $(foreach elf, ${ELFS}, ${OBJCOPY} ${elf} --strip-all -O binary $(patsubst ${TARGET_DIR}/%, ${TARGET_DIR}/%.bin, ${elf});)

build: binary

clean:
    @cargo clean

.PHONY: elf binary build clean
```

(2)增加 yield 系统调用

- 多道程序操作系统中，用户程序可以主动让出 CPU，让其他应用程序继续执行。因此需要先在用户态系统调用封装中增加 `sys_yield`。

Bash

```
1 vim user/src/syscall.rs
```

在系统调用编号处增加：

```
const SYSCALL_WRITE: usize = 64;
const SYSCALL_EXIT: usize = 93;
const SYSCALL_YIELD: usize = 124;
```

然后在文件末尾增加：

```
pub fn sys_yield() -> isize {
    syscall(SYSCALL_YIELD, [0, 0, 0])
}
```

25,1

Bot

- 修改 `user/src/lib.rs`，在用户库中继续封装 `yield_`。这样用户程序就可以直接调用 `yield_()` 主动让出 CPU。

Bash

```
1 vim user/src/lib.rs
```

在 `exit` 函数后面加入：

```
pub fn exit(exit_code: i32) -> isize {
    sys_exit(exit_code)
}

pub fn yield_() -> isize {
    sys_yield()
}
```

(3)实现测试应用程序

先删除实验 3 中的旧应用程序，重新创建实验 4 所需的三个测试程序。

Bash

```
1 rm user/src/bin/*.rs
2 vim user/src/bin/00write_a.rs
```

写入以下内容

```
[root@7b99f7bbef25 mnt]# cat user/src/bin/00write_a.rs
#![no_std]
#![no_main]
#[macro_use]
extern crate user_lib;

use user_lib::yield_;

const WIDTH: usize = 10;
const HEIGHT: usize = 5;

#[unsafe(no_mangle)]
fn main() -> i32 {
    for i in 0..HEIGHT {
        for _ in 0..WIDTH {
            print!("{}", "A");
        }
        println!("{}", [{} / {}], i + 1, HEIGHT);
        yield_();
    }
    println!("Test write_a OK!");
    0
}
```

Bash

```
1 vim user/src/bin/01write_b.rs
```

写入以下内容：

```
@7b99f7bbef25:/mnt
"user/src/bin/01write_b.rs" [New] 0,0-1 All
~
~
#![no_std]
#![no_main]
#![macro_use]
extern crate user_lib;

use user_lib::yield_;

const WIDTH: usize = 10;
const HEIGHT: usize = 5;

#[unsafe(no_mangle)]
fn main() -> i32 {
    for i in 0..HEIGHT {
        for _ in 0..WIDTH {
            print!("{}", "B");
        }
        println!("{}", [{} / {}], i + 1, HEIGHT);
        yield_();
    }
    println!("Test write_b OK!");
    0
}
```

Bash

```
1 vim user/src/bin/02write_c.rs
```

写入以下内容：

```
@7b99f7bbef25:/mnt
"user/src/bin/02write_c.rs" 22L, 373B 22,1 All
~
~
#![no_std]
#![no_main]
#![macro_use]
extern crate user_lib;

use user_lib::yield_;

const WIDTH: usize = 10;
const HEIGHT: usize = 5;

#[unsafe(no_mangle)]
fn main() -> i32 {
    for i in 0..HEIGHT {
        for _ in 0..WIDTH {
            print!("{}", "C");
        }
        println!("{}", [{} / {}], i + 1, HEIGHT);
        yield_();
    }
    println!("Test write_c OK!");
    0
}
```

- 编译三个用户程序。此时 `build.py` 会为不同应用程序设置不同的链接地址，并生成对应的 `.bin` 文件。

Bash

```
1 cd user
2 make build
```

```
[root@7b99f7bbef25 mnt]# cd user
make build
Compiling user_lib v0.1.0 (/mnt/user)
Finished `release` profile [optimized] target(s) in 0.37s
[build.py] application 00write_a start with address 0x80400000
Compiling user_lib v0.1.0 (/mnt/user)
Finished `release` profile [optimized] target(s) in 0.03s
[build.py] application 01write_b start with address 0x80420000
Compiling user_lib v0.1.0 (/mnt/user)
Finished `release` profile [optimized] target(s) in 0.02s
[build.py] application 02write_c start with address 0x80440000
rust-objcopy --binary-architecture=riscv64 target/riscv64gc-unknown-none-elf/release/00write_a --strip-all -O binary target/riscv64gc-unknown-none-elf/release/00write_a.bin; rust-objcopy --binary-architecture=riscv64 target/riscv64gc-unknown-none-elf/release/01write_b --strip-all -O binary target/riscv64gc-unknown-none-elf/release/01write_b.bin; rust-objcopy --binary-architecture=riscv64 target/riscv64gc-unknown-none-elf/release/02write_c --strip-all -O binary target/riscv64gc-unknown-none-elf/release/02write_c.bin;
[root@7b99f7bbef25 user]#
```

2. 多道程序的加载

- 将实验 3 中分散在 `batch.rs` 里的常量单独放到 `config.rs` 中，方便后续 `loader` 和 `task` 模块共同使用。

Bash

```
1 cd /mnt
2 vim os/src/config.rs
```

写入以下内容

```
"os/src/config.rs" [New] 0,0-1 All
pub const USER_STACK_SIZE: usize = 4096 * 2;
pub const KERNEL_STACK_SIZE: usize = 4096 * 2;
pub const MAX_APP_NUM: usize = 4;
pub const APP_BASE_ADDRESS: usize = 0x80400000;
pub const APP_SIZE_LIMIT: usize = 0x20000;
```

- 创建 `loader.rs`，实现多道程序使用的用户栈和内核栈。与实验 3 不同，这里为每个应用程序都准备一组用户栈和内核栈。

Bash

```
1 vim os/src/loader.rs
```

写入以下内容：

```
"os/src/loader.rs" [New] 0,0-1 All
use crate::task::TaskContext;
#[derive(Copy, Clone)]
struct KernelStack {
    data: [u8; KERNEL_STACK_SIZE],
}

static USER_STACK: [UserStack; MAX_APP_NUM] = [
    UserStack { data: [0; USER_STACK_SIZE] },
    MAX_APP_NUM
];

impl KernelStack {
    fn get_sp(&self) -> usize {
        self.data.as_ptr() as usize + KERNEL_STACK_SIZE
    }
}
```

```
pub fn push_context(&self, trap_cx: TrapContext, task_cx: TaskContext) -> &'static mut TaskContext {
    unsafe {
        let trap_cx_ptr = (self.get_sp() - core::mem::size_of::<TrapContext>()) as *mut TrapContext;
        *trap_cx_ptr = trap_cx;
        let task_cx_ptr = (trap_cx_ptr as usize - core::mem::size_of::<TaskContext>()) as *mut TaskContext;
        *task_cx_ptr = task_cx;
        task_cx_ptr.as_mut().unwrap()
    }
}

impl UserStack {
    fn get_sp(&self) -> usize {
        self.data.as_ptr() as usize + USER_STACK_SIZE
    }
}
```

48,1 Bot

- 继续在 `loader.rs` 中加入应用程序加载逻辑。内核启动时会一次性把所有应用程序加载到不同的内存地址，每个应用程序的地址由 `APP_BASE_ADDRESS + app_id * APP_SIZE_LIMIT` 计算得到。

Bash

```
1 vim os/src/loader.rs
```

在文件末尾继续加入以下内容：

```
fn get_base_i(app_id: usize) -> usize {
    APP_BASE_ADDRESS + app_id * APP_SIZE_LIMIT
}

pub fn get_num_app() -> usize {
    unsafe extern "C" {
        fn _num_app();
    }
    unsafe {
        (_num_app as *const ()) as *const usize).read_volatile()
    }
}

pub fn load_apps() {
    unsafe extern "C" {
        fn _num_app();
    }
    let num_app_ptr = _num_app as *const () as *const usize;
    let num_app = get_num_app();
```

```
    let app_start = unsafe {
        core::slice::from_raw_parts(num_app_ptr.add(1), num_app + 1)
    };
    unsafe {
        asm!("fence.i");
    }
    for i in 0..num_app {
        let base_i = get_base_i(i);
        (base_i..base_i + APP_SIZE_LIMIT).for_each(|addr| unsafe {
            (addr as *mut u8).write_volatile(0)
        });
        let src = unsafe {
            core::slice::from_raw_parts(app_start[i] as *const u8, app_start[i + 1] - app_start[i])
        };
        let dst = unsafe {
            core::slice::from_raw_parts_mut(base_i as *mut u8, src.len())
        };
        dst.copy_from_slice(src);
    }
}
```

87,1 Bot

3.任务的设计与实现

(1)任务的上下文

- 创建 `task` 模块目录，并定义 `TaskContext`。任务切换时需要保存恢复任务继续执行所需的寄存器，这里主要保存返回地址 `ra` 和被调用者保存寄存器 `s0-s11`。

Bash

```
1 mkdir -p os/src/task
2 vim os/src/task/context.rs
```

写入以下内容：

```
@7b99f7bbef25:/mnt
~
"os/src/task/context.rs" 17L, 301B
#[repr(C)]
pub struct TaskContext {
    ra: usize,
    s: [usize; 12],
}

impl TaskContext {
    pub fn goto_restore() -> Self {
        unsafe extern "C" {
            fn __restore();
        }
        Self {
            ra: __restore as *const () as usize,
            s: [0; 12],
        }
    }
}
```

(2)任务的运行状态及任务控制块

- 创建 `task.rs`，定义任务运行状态和任务控制块。任务控制块中保存任务上下文指针和任务状态，后续调度器会根据这些信息切换不同任务。

Bash

```
1 vim os/src/task/task.rs
```

写入以下内容：

```
@7b99f7bbef25:/mnt
~
"os/src/task/task.rs" 19L, 351B
#[derive(Copy, Clone, PartialEq)]
pub enum TaskStatus {
    UnInit,
    Ready,
    Running,
    Exited,
}

#[derive(Copy, Clone)]
pub struct TaskControlBlock {
    pub task_cx_ptr: usize,
    pub task_status: TaskStatus,
}

impl TaskControlBlock {
    pub fn get_task_cx_ptr2(&self) -> *const usize {
        &self.task_cx_ptr as *const usize
    }
}
```

(3)任务切换

- 创建 `switch.S`，实现任务切换的汇编函数 `__switch`。它会保存当前任务的 `ra` 和 `s0-s11`，再恢复下一个任务的上下文，从而完成任务切换。

Bash

```
1 vim os/src/task/switch.S
```

写入以下内容：

```
@7b99f7bbef25:/mnt
.altmacro

.macro SAVE_SN n
    sd s\@n, (\@n+1)*8(sp)
.endm

.macro LOAD_SN n
    ld s\@n, (\@n+1)*8(sp)
.endm

.section .text
.globl __switch

__switch:
    addi sp, sp, -13*8
    sd sp, 0(a0)
    sd ra, 0(sp)
    .set n, 0
    .rept 12
        SAVE_SN \@n
        .set n, n + 1
    .endr
    ld sp, 0(a1)
    ld ra, 0(sp)
    .set n, 0
    .rept 12
        LOAD_SN \@n
        .set n, n + 1
    .endr
    addi sp, sp, 13*8
    ret
```

- 创建 `switch.rs`，把汇编中的 `__switch` 封装成 Rust 可以调用的函数。

Bash

```
1 vim os/src/task/switch.rs
```

写入以下内容：

```
@7b99f7bbef25:/mnt
"os/src/task/switch.rs" [New] 0,0-1 All
use core::arch::global_asm;

global_asm!(include_str!("switch.S"));

unsafe extern "C" {
    pub fn __switch(
        current_task_cx_ptr2: *const usize,
        next_task_cx_ptr2: *const usize,
    );
}
```

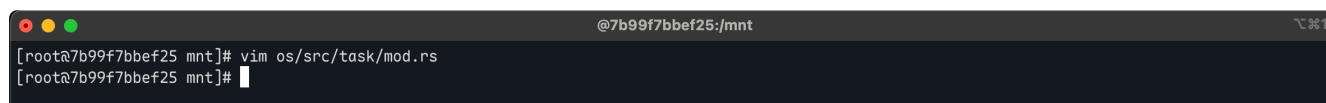
(4)任务管理器

- 创建 `task/mod.rs`，实现任务管理器。它负责初始化所有任务、记录当前任务、查找下一个可运行任务，并在任务主动让出 CPU 或退出时切换到下一个任务。

Bash

```
1 vim os/src/task/mod.rs
```

具体代码略，见 git 仓库



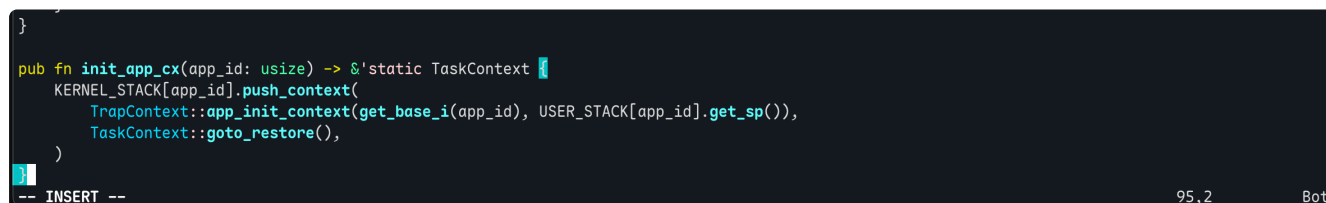
```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/task/mod.rs
[root@7b99f7bbef25 mnt]#
```

- 继续补充 `loader.rs`，实现 `init_app_cx`。该函数会为指定应用程序构造初始上下文：先压入用户态 Trap 上下文，再压入任务上下文，供任务管理器第一次调度时使用。

Bash

```
1 vim os/src/loader.rs
```

在文件末尾加入以下内容：



```
}
pub fn init_app_cx(app_id: usize) -> &'static TaskContext {
    KERNEL_STACK[app_id].push_context(
        TrapContext::app_init_context(get_base_i(app_id), USER_STACK[app_id].get_sp()),
        TaskContext::goto_restore(),
    )
}
-- INSERT --
```

4.实现 sys_yield 和 sys_exit 系统调用

- 修改 `process.rs`，让 `sys_exit` 不再直接运行下一个应用，而是通过任务管理器标记当前任务退出并切换任务。同时新增 `sys_yield`，用于主动让出 CPU。

Bash

```
1 vim os/src/syscall/process.rs
```

将文件修改为以下内容：

```
@7b99f7bbef25:/mnt
"os/src/syscall/process.rs" 7L, 163B 7,0-1 ALL
use crate::task::{
    exit_current_and_run_next,
    suspend_current_and_run_next,
};

pub fn sys_exit(exit_code: i32) -> ! {
    println!("[kernel] Application exited with code {}", exit_code);
    exit_current_and_run_next();
    panic!("Unreachable in sys_exit!");
}

pub fn sys_yield() -> isize {
    suspend_current_and_run_next();
    0
}
```

- 修改 `syscall/mod.rs`，加入 `yield` 系统调用编号，并在系统调用分发中处理 `SYSCALL_YIELD`。

Bash

```
1 vim os/src/syscall/mod.rs
```

将文件修改为以下内容：

```
@7b99f7bbef25:/mnt
const SYSCALL_WRITE: usize = 64;
const SYSCALL_EXIT: usize = 93;
const SYSCALL_YIELD: usize = 124;

mod fs;
mod process;

use fs::*;
use process::*;

pub fn syscall(syscall_id: usize, args: [usize; 3]) -> isize {
    match syscall_id {
        SYSCALL_WRITE => sys_write(args[0], args[1] as *const u8, args[2]),
        SYSCALL_EXIT => sys_exit(args[0] as i32),
        SYSCALL_YIELD => sys_yield(),
        _ => panic!("Unsupported syscall_id: {}", syscall_id),
    }
}
```

5.修改其他部分代码

- 实验 4 不再通过 `batch::run_next_app()` 直接运行下一个应用，而是通过 `task` 模块进行任务切换。因此需要修改 `trap/mod.rs`，去掉对 `run_next_app` 的调用。

Bash

```
1 vim os/src/trap/mod.rs
```

注释以下代码行：

```
};
// use crate::batch::run_next_app;
use crate::syscall::syscall;
```

```

Trap::Exception(Exception::StoreFault) | Trap::Exception(Exception::StorePageFault) => {
    println!("[kernel] PageFault in application, core dumped.");
    run_next_app();
}
Trap::Exception(Exception::IllegalInstruction) => {
    println!("[kernel] IllegalInstruction in application, core dumped.");
    run_next_app();
}
=> {

```

- 修改 `trap.S` 中的 `__restore`。实验 4 中任务切换会保证 `sp` 已经指向正确的任务上下文位置

Bash

```
1 vim os/src/trap/trap.S
```

注释以下代码行：

```

__restore:
# mv sp, a0
ld t0, 32*8(sp)
ld t1, 33*8(sp)
ld t2, 2*8(sp)

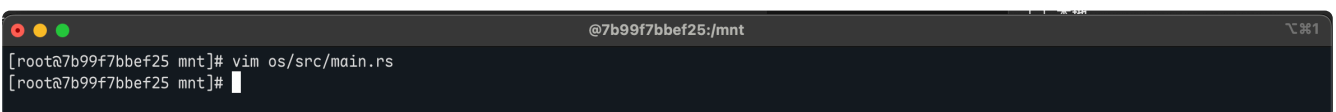
```

- 修改 `main.rs`，引入 `loader`、`config`、`task` 模块。内核启动后先加载所有应用程序，再运行第一个任务，由任务管理器负责后续调度。

Bash

```
1 vim os/src/main.rs
```

具体代码略，见 git 仓库



```

@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/main.rs
[root@7b99f7bbef25 mnt]#

```

6.编译并运行

先重新编译用户程序。实验 4 的 `build.py` 会分别为 `00write_a`、`01write_b`、`02write_c` 设置不同的加载地址，并生成对应的 `.bin` 文件。

Bash

```

1 cd /mnt/user
2 make build

```

```
@7b99f7bbef25:/mnt/user
[root@7b99f7bbef25 mnt]# vim os/src/main.rs
[root@7b99f7bbef25 mnt]# cd /mnt/user
make build
    Finished `release` profile [optimized] target(s) in 0.01s
[build.py] application 00write_a start with address 0x80400000
    Finished `release` profile [optimized] target(s) in 0.00s
[build.py] application 01write_b start with address 0x80420000
    Finished `release` profile [optimized] target(s) in 0.00s
[build.py] application 02write_c start with address 0x80440000
rust-objcopy --binary-architecture=riscv64 target/riscv64gc-unknown-none-elf/release/00write_a --strip-all -O binary target/riscv64gc-unknown-none-elf/release/00write_a.bin; rust-objcopy --binary-architecture=riscv64 target/riscv64gc-unknown-none-elf/release/01write_b --strip-all -O binary target/riscv64gc-unknown-none-elf/release/01write_b.bin; rust-objcopy --binary-architecture=riscv64 target/riscv64gc-unknown-none-elf/release/02write_c --strip-all -O binary target/riscv64gc-unknown-none-elf/release/02write_c.bin;
[root@7b99f7bbef25 user]#
```

编译并运行内核：

Bash

- 1 `cd /mnt/os`
- 2 `make run`

```
@7b99f7bbef25:/mnt/os
Firmware Size      : 120 KB
Runtime SBI Version : 0.2

MIDELEG : 0x0000000000000222
MEDELEG : 0x000000000000b109
PMP0     : 0x0000000000000000-0x0000000000001ffff (A)
PMP1     : 0x0000000000000000-0xffffffffffffffff (A,R,W,X)
[kernel] Hello, world!
AAAAAAAAAA [1/5]
BBBBBBBBBB [1/5]
CCCCCCCCCC [1/5]
AAAAAAAAAA [2/5]
BBBBBBBBBB [2/5]
CCCCCCCCCC [2/5]
AAAAAAAAAA [3/5]
BBBBBBBBBB [3/5]
CCCCCCCCCC [3/5]
AAAAAAAAAA [4/5]
BBBBBBBBBB [4/5]
CCCCCCCCCC [4/5]
AAAAAAAAAA [5/5]
BBBBBBBBBB [5/5]
CCCCCCCCCC [5/5]
Test write_a OK!
[kernel] Application exited with code 0
Test write_b OK!
[kernel] Application exited with code 0
Test write_c OK!
[kernel] Application exited with code 0
Panicked at src/task/mod.rs:93 All applications completed!
```

至此，支持协作式调度的多道程序操作系统实现完成。

三.问题回答

1.分析应用程序是如何加载的

实验 4 中，每个应用程序不再加载到同一个地址，而是通过 build.py 为不同应用程序设置不同的链接地址。

Python

```
1 base_address = 0x80400000
2 step = 0x20000
3 line = line.replace(hex(base_address), hex(base_address + step * app_i
    d))
```

这样 `00write_a`、`01write_b`、`02write_c` 会分别被链接到不同的地址，避免多个应用程序在内存中互相覆盖。

编译完成后，内核通过 `link_app.S` 将应用程序二进制码链接进内核镜像。

Bash

```
1 app_0_start:
2     .incbin "../user/target/riscv64gc-unknown-none-elf/release/00write_
    a.bin"
3 app_0_end:
```

其中 `app_0_start` 和 `app_0_end` 标记了应用程序在内核镜像中的起止位置，内核可以根据这些符号找到每个应用程序。

内核启动后调用 `loader::load_apps()`，把所有应用程序一次性加载到对应内存位置。

Rust

```
1 let base_i = APP_BASE_ADDRESS + i * APP_SIZE_LIMIT;
2 let dst = core::slice::from_raw_parts_mut(base_i as *mut u8, src.len
    ());
3 dst.copy_from_slice(src);
```

2.分析多道程序如何设计和实现的

实验 4 将实验 3 中的批处理结构拆分为 `loader` 和 `task` 两部分。`loader` 负责加载应用程序，`task` 负责管理和切换任务。

Rust

```
1 loader::load_apps();
2 task::run_first_task();
```

内核启动后先加载所有应用程序，然后运行第一个任务，之后由任务管理器负责调度。

多道程序通过任务管理器维护所有任务的状态。每个任务都有自己的任务控制块，用来保存任务上下文地址和任务状态。

Rust

```
1 pub struct TaskControlBlock {
2     pub task_cx_ptr: usize,
3     pub task_status: TaskStatus,
4 }
```

应用程序通过 `yield_()` 主动让出 CPU，内核收到 `sys_yield` 系统调用后切换到下一个任务。

Rust

```
1 pub fn sys_yield() -> isize {
2     suspend_current_and_run_next();
3     0
4 }
```

3.分析所实现的多道程序操作系统中的任务是如何实现的，以及它和理论课程里的进程和线程有什么区别和联系。

(1) 任务切换需要保存任务上下文。本实验中 TaskContext 保存返回地址 ra 和 s0-s11 寄存器。

Rust

```
1 pub struct TaskContext {
2     ra: usize,
3     s: [usize; 12],
4 }
```

这些寄存器足够让任务在被切换出去后，之后还能从原来的位置继续执行。

真正的任务切换由汇编函数 `__switch` 完成。它先保存当前任务上下文，再恢复下一个任务上下文。

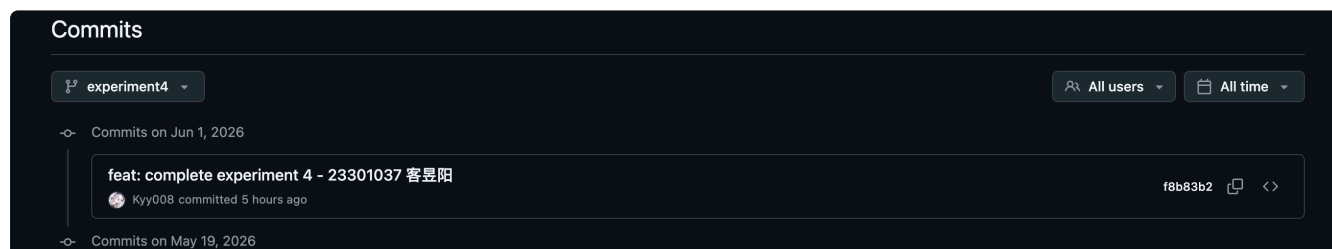
Rust

```
1 sd ra, 0(sp)
2 ld sp, 0(a1)
3 ld ra, 0(sp)
4 ret
```

这样 CPU 的执行流就从当前任务切换到了下一个任务。

(2) 本实验中的任务和理论课中的进程、线程有联系，但更加简化。它像线程一样拥有自己的执行上下文和栈，可以被调度和切换；但它不像完整进程那样拥有独立地址空间、文件资源和复杂的进程管理结构。因此，本实验中的任务可以理解为一个简化的用户程序执行单元。它具备任务上下文、运行状态和调度切换能力，但还没有实现完整进程的资源隔离，也没有实现基于时钟中断的抢占式调度。

四.Git 提交截图



五.其他说明

无